

AULA 5

Integração com Banco de Dados

Disciplina: CK0207 — Desenvolvimento de Software para Web

Professor: Dr. Fernando Antônio Mota Trinta

Universidade Federal do Ceará (UFC)

Projeto do curso: `nest-api-curso` (continuação da Aula 4)

Autor: Samuel Sales Furtado

Objetivos de Aprendizagem

- Entender o que um ORM resolve e por que usar um
- Instalar e inicializar o Prisma num projeto NestJS
- Modelar uma entidade no schema.prisma
- Criar e aplicar sua primeira migration
- Entender quando usar SQLite e quando usar PostgreSQL

Esta é uma aula mais conceitual: ao final, o banco de dados existirá e estará migrado, mas o array em memória do `UserService` só é efetivamente substituído pelo Prisma na **Aula 6** (Laboratório: Persistência de Dados).

1. Conceitos de ORM

Um ORM (*Object-Relational Mapper*) permite trabalhar com o banco de dados manipulando objetos e classes, em vez de escrever SQL puro na mão. Ele cuida de três coisas: gerar as queries por trás dos panos, mapear as linhas retornadas para objetos tipados, e gerenciar migrations — o histórico versionado da evolução do schema do banco.

Trade-offs: a vantagem é produtividade, tipagem e portabilidade entre bancos. O cuidado é que um ORM é uma camada de abstração a mais — usada sem atenção, pode esconder queries pouco eficientes. O NestJS não força nenhum ORM específico: integra-se bem com TypeORM, Sequelize, Mongoose e Prisma. Este curso usa **Prisma**, pela tipagem forte gerada automaticamente a partir do schema.

2. Introdução ao Prisma ORM

```
npm install prisma --save-dev
npm install @prisma/client

npx prisma init --datasource-provider sqlite
```

Esse comando gera três arquivos:

Arquivo	Papel
<code>schema.prisma</code>	Schema do banco: models + configuração
<code>prisma.config.ts</code>	Configuração do Prisma (schema, migrations, datasource)
<code>.env</code>	Variáveis de ambiente, incluindo <code>DATABASE_URL</code>

Nota sobre a versão atual do Prisma: desde a versão 7, cada banco de dados exige um pacote *driver adapter* específico para se conectar — por exemplo, `@prisma/adapter-better-sqlite3` para SQLite, ou `@prisma/adapter-pg` para PostgreSQL. Isso difere de versões mais antigas do Prisma, que traziam um motor único embutido para todos os bancos. Este curso usa a sintaxe atual; a conexão efetiva do adapter dentro do NestJS é implementada na Aula 6.

3. Modelagem de Entidades

```
generator client {
  provider = "prisma-client"
  output   = "../generated/prisma"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model User {
  id      Int    @id @default(autoincrement())
  name    String
  email   String @unique
}
```

O `model User` é, essencialmente, o mesmo `User` usado desde a Aula 2 — agora com um schema formal, entendido pelo banco de dados, não apenas pelo TypeScript.

- `@id` marca a chave primária.
- `@default(autoincrement())` faz o banco gerar o id sozinho — o mesmo papel que o `idCounter` manual desempenhava até aqui, agora implementado de forma confiável pelo próprio banco.
- `@unique` em `email` impede dois usuários com o mesmo e-mail — e essa garantia passa a vir do **banco de dados**, não apenas de uma checagem manual no código.

4. Migrations

```
npx prisma migrate dev --name init
```

Esse comando: compara o schema atual com o estado anterior do banco, gera um arquivo SQL com exatamente as mudanças necessárias, aplica esse SQL no banco de desenvolvimento, e roda `prisma generate` automaticamente ao final, atualizando o Prisma Client com os tipos mais recentes.

Migrations são arquivos SQL versionados junto com o código (normalmente em `prisma/migrations`), garantindo que todo ambiente — máquina local, de um colega, ou servidor de produção — aplique exatamente o mesmo histórico de mudanças, na mesma ordem. **Regra prática:** toda vez que o `schema.prisma` mudar, o próximo passo é rodar `migrate dev` de novo — nunca editar o banco manualmente por fora do Prisma.

5. PostgreSQL e SQLite

	SQLite	PostgreSQL
Infraestrutura	Arquivo local, sem servidor	Requer servidor de banco rodando
Uso recomendado	Aprendizado e prototipagem rápida	Produção, com concorrência e carga real
Driver adapter	<code>@prisma/adapter-better-sqlite3</code>	<code>@prisma/adapter-pg</code>

Trocar de um banco para o outro no Prisma é uma mudança pequena: o `provider` no `datasource` muda, a `DATABASE_URL` muda de um caminho de arquivo para uma string de conexão, e o driver adapter troca de pacote. O restante do código — `models` e `queries` — permanece igual.

Laboratório — Passo a Passo

Ao final deste laboratório, o banco existirá e estará migrado — a conexão do Prisma dentro do código do NestJS acontece na Aula 6.

Passo 1 — Instalar o Prisma

```
npm install prisma --save-dev
npm install @prisma/client
```

Passo 2 — Inicializar com SQLite

```
npx prisma init --datasource-provider sqlite
```

Testar: confirmar que `prisma/schema.prisma`, `.env` e `prisma.config.ts` foram criados.

Passo 3 — Editar o `schema.prisma`

```
generator client {
  provider = "prisma-client"
  output   = "../generated/prisma"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model User {
  id      Int      @id @default(autoincrement())
  name    String
  email   String @unique
}
```

Conferir o `.env`:

```
DATABASE_URL="file:./dev.db"
```

Passo 4 — Criar e aplicar a primeira migration

```
npx prisma migrate dev --name init
```

Testar: `ls prisma/migrations` deve mostrar uma pasta com o SQL gerado. Confirmar também que `dev.db` foi criado na raiz do projeto.

Passo 5 — Explorar o banco (opcional)

```
npx prisma studio
```

Abre uma interface visual no navegador para ver a tabela `User`, ainda vazia — os dados por código chegam na Aula 6.

Verificação de Conformidade

- ✓ Instalação e `npx prisma init` — conferido em docs.nestjs.com/recipes/prisma e prisma.io/docs.
- ✓ Bloco `generator client` com `output` customizado — conferido em docs.nestjs.com/recipes/prisma.
- ✓ Necessidade de driver adapter por banco na versão atual do Prisma — conferido em múltiplas páginas oficiais do Prisma sobre adaptadores.
- ✓ `@id`, `@default(autoincrement())`, `@unique` — conferido no Prisma Schema Reference oficial.
- ✓ `npx prisma migrate dev` gerando SQL e aplicando no banco — conferido em docs.nestjs.com/recipes/prisma.